Assignement 1 :

The **DBpedia Endpoint** is an online service that allows users to query the **DBpedia knowledge graph** using the **SPARQL query language**.

DBpedia is a large, structured knowledge base created by extracting information from Wikipedia and representing it as **RDF triples** (subject–predicate–object)

Why is the DBpedia Endpoint useful?

The DBpedia Endpoint is useful because it allows applications to:

- Retrieve structured information from Wikipedia.
- Explore relationships between entities.
- Build semantic web and linked data applications.
- Answer complex questions using SPARQL instead of manually reading Wikipedia pages.

Line 1

```
PREFIX dbr: <http://dbpedia.org/resource/>
```

What does it do?

This line defines a **prefix** (a shortcut).

Instead of writing the full URI every time:

```
http://dbpedia.org/resource/Jennifer_Aniston
```

we can simply write:

```
dbr:Jennifer_Aniston
```

Here:

- **dbr** stands for **DBpedia Resource**
- It refers to actual entities (people, places, movies, companies, etc.)

For example:

```
dbr:Jennifer_Aniston
dbr:Brad_Pitt
dbr:Italy
dbr:Google
```

These are all **resources** in DBpedia.

Line 2

```
PREFIX dbo: <http://dbpedia.org/ontology/>
```

This defines another shortcut.

Instead of writing

```
http://dbpedia.org/ontology/spouse
```

we simply write

```
dbo:spouse
```

Here:

- **dbo** stands for **DBpedia Ontology**
- It contains properties (relationships or attributes).

Examples include:

```
dbo:spouse
dbo:birthPlace
dbo:author
dbo:director
dbo:capital
```

These describe relationships between resources.

A simple way to remember it is:

- **dbr = Things**
- **dbo = Relationships**

Line 3

- `SELECT ?spouse`
- This tells SPARQL:
- "I want the value stored in the variable `?spouse`."
- The question mark (`?`) indicates a **variable**.
- Just like a variable in Python:
- `x`
- In SPARQL we write:
- `?spouse`
- This variable will hold the answer.

- # Line 4
- `WHERE`
- This introduces the search condition.
- It means:
- "Find data that satisfies the following pattern."
- # Line 5
- `{`
- This begins the graph pattern (the search conditions).
- # The most important line
- `dbr:Jennifer_Aniston dbo:spouse ?spouse .`
- This line is an **RDF Triple Pattern**.
- It has exactly three parts:

Subject	Predicate	Object
dbr:Jennifer_Aniston	dbo:spouse	?spouse

- Let's examine each one.

Query 2 :

```
PREFIX dbr: <http://dbpedia.org/resource/>

PREFIX dbo: <http://dbpedia.org/ontology/>


SELECT (COUNT(?film) AS ?numFilms) WHERE {

 ?film dbo:starring dbr:Jennifer_Aniston .

}
```

`PREFIX dbr:`

- Defines the prefix for DBpedia resources (entities), such as `Jennifer_Aniston`.

`PREFIX dbo:`

- Defines the prefix for DBpedia ontology properties, such as `starring`.

`SELECT (COUNT(?film) AS ?numFilms)`

- Instead of returning every matching film, this query counts them.
- `COUNT(?film)` counts the number of films found.
- `AS ?numFilms` stores the result in a variable named `?numFilms`.

```
WHERE {
```

- Starts the graph pattern that specifies what to search for.

```
?film dbo:starring dbr:Jennifer_Aniston .
```

- `?film` is a variable representing any film.
- `dbo:starring` is the relationship meaning "features an actor."
- `dbr:Jennifer_Aniston` is the specific actor we are searching for.
- In other words, the query looks for every film in DBpedia where Jennifer Aniston is listed as a starring actor.

```
}
```

- Ends the `WHERE` clause.

Query 3 :

5. Give me the total number of actors that performed in the latest film of your favorite actor.

```
PREFIX dbr: <http://dbpedia.org/resource/>

PREFIX dbo: <http://dbpedia.org/ontology/>


SELECT (COUNT(?actor) AS ?numActors) WHERE {

  ?film dbo:starring dbr:Jennifer_Aniston ;

      dbo:releaseDate ?date ;

      dbo:starring ?actor .

}

ORDER BY DESC(?date)
```

- **`PREFIX dbr:`** Defines the prefix for DBpedia resources, such as `Jennifer_Aniston`.
- **`PREFIX dbo:`** Defines the prefix for DBpedia ontology properties.

- **`SELECT (COUNT(?actor) AS ?numActors)`** Counts the number of actors and stores the result in the variable `?numActors`.
- **`?film dbo:starring dbr:Jennifer_Aniston`** Finds films in which Jennifer Aniston starred.
- **`dbo:releaseDate ?date`** Retrieves the release date of each film.
- **`dbo:starring ?actor`** Retrieves every actor who starred in each film.
- **`ORDER BY DESC(?date)`** Sorts the films from newest to oldest.

Limist 1 : **Keep only the first result**, which is the **most recently released film**.

So the intended logic becomes:

1. Find all films starring Jennifer Aniston.
2. Retrieve the release date of each film.
3. Sort the films by release date in descending order (newest first).
4. **Keep only the latest film** (`LIMIT 1`).
5. Count the actors who starred in that film.

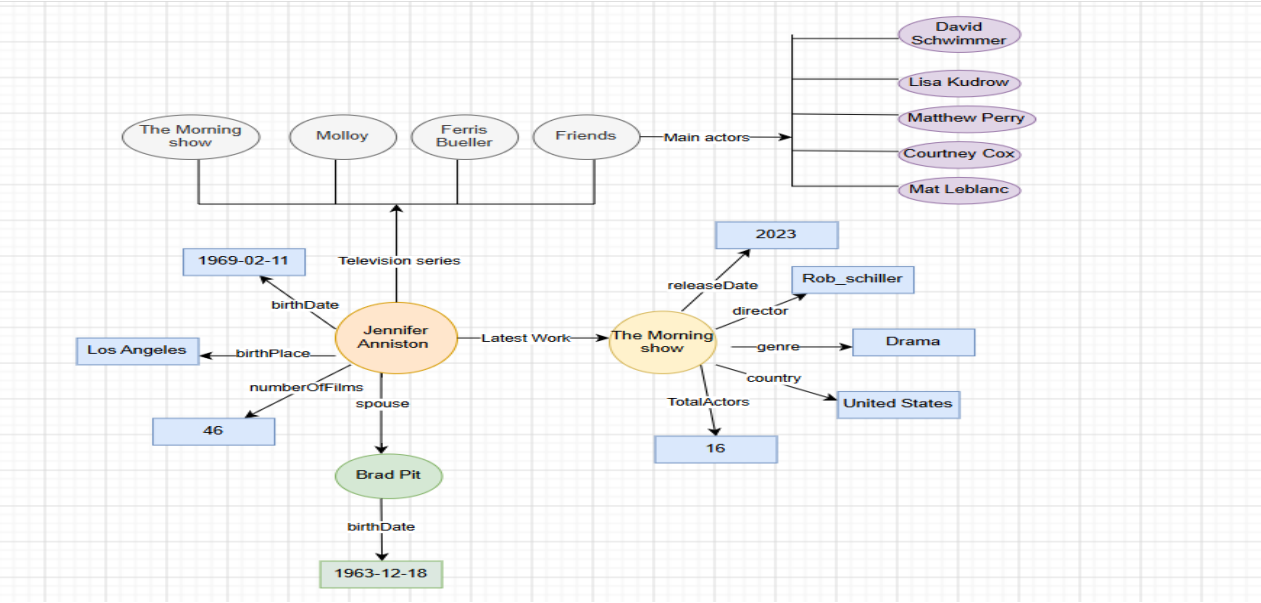
Explaining the RDF Graph :

Your graph is actually well structured. It follows the RDF idea of:

Subject → Predicate → Object

where:

- **Subject** = the main entity (circle)
- **Predicate** = the relationship (arrow label)
- **Object** = another entity or a literal value



- **The circles represent entities (or nodes) in the knowledge graph, such as people and TV series. The rectangles represent literal values or attributes, such as dates, numbers, genres, and countries. The arrows represent relationships (predicates) between nodes, and the labels on the arrows indicate the type of relationship, such as *spouse*, *birthPlace*, or *director*. Overall, the graph follows the RDF structure of Subject–Predicate–Object, where entities are connected through semantic relationships or linked to their attribute values.**

Assignement 2 :

The easiest way to determine **Domain** and **Range** is to ask two simple questions about the relationship (property):

1. **Who can have this property? → Domain**
2. **What kind of value does this property point to? → Range**

In other words:

- **Domain** = the class of the **subject** (where the relationship starts).
- **Range** = the class of the **object** (where the relationship ends).

Question : for the names, why we use data properties ?

We use a data property because a person's name **is a literal value (text)**, not another entity in the ontology. Object properties are used to connect one entity (individual) to another entity, whereas data properties connect an entity to a data value such as a string, number, or date. Since a name is simply a text value, its range is `xsd:string`, making `hasName` a data property rather than an object property.

Assignement 3 :

۱) batch_size .تعداد جملات هم‌زمان

- **تعریف ساده :**مشخص می‌کند که مدل در آنِ واحد، دارد به چند جمله‌ی ورودیِ مختلف فکر می‌کند و پاسخ می‌دهد.

۲) sequence_length .طول جمله یا تعداد کلمات)

- **تعریف ساده :**تعداد کل کلمات (توکن‌های) موجود در جمله‌ای است که شما به مدل داده‌اید.

۳) vocab_size .اندازه کل واژه‌نامه مدل)

- **تعریف ساده :**کل کلمات، حروف و علائمی است که این هوش مصنوعی در طول زندگی‌اش یاد گرفته و بلد است آن‌ها را بنویسد.

Token : In the context of Natural Language Processing (NLP) and Large Language Models (like the GPT-2 model used in your assignment), a **token** is the fundamental unit of data processed by the model.

- **`tokenizer(...)`**: Translates your text string into a list of numbers (IDs) that represent those words. `return_tensors='pt'` tells it to format these numbers specifically for PyTorch.

در بالاترین بخش، دو عدد) شناسه یا (ID تعریف شده‌اند که نشان‌دهنده خط جدید (`198`) و پایان متن (`50256`) هستند. اگر مدل به این توکن‌ها برسد، یعنی کارش تمام شده و باید ایست کند.

تبدیل متن به عدد :(Tokenization) در خط اولِ تابع، متن ورودی شما به شماره‌های عددی قابل فهم برای مدل (`input_ids`)تبدیل می‌شود.

•ورود به حلقه تولید :(For Loop) این حلقه به تعداد `max_new_tokens`(در اینجا حداکثر ۱۰۰ بار) تکرار می‌شود تا کلمات جدید را یکی‌یکی بسازد:

1. **اجرای مدل :(`model(...)`)** متن عددی‌شده به مدل داده می‌شود و مدل خروجی پیش‌بینی‌ها (همان `output.logits`که صحبتش را کردیم) را تحویل می‌دهد.
2. **جدا کردن نمرات آخرین کلمه :(`[:, -1, :]`)** از بین تمام کلمات، مدل فقط به نمرات کلمه **آخر** نیاز دارد تا بر اساس آن کلمه بعدی را حدس بزند.
3. **انتخاب بهترین کلمه :(`torch.argmax`)** متد `argmax`بالاترین نمره خام (Logit) را پیدا می‌کند. یعنی مدام کلمه‌ای را انتخاب می‌کند که مدل بیشترین اطمینان را به آن دارد (به این روش جستجوی حریصانه یا **Greedy Search**می‌گویند.)
4. **چسباندن کلمه جدید به متن قبلی :(`torch.cat`)** کلمه جدیدِ تولید شده به انتهای متن قبلی چسبانده می‌شود تا در دور بعدی حلقه، مدل متن طولانی‌تر را ببیند.
5. **شرط خروج :(`if next_token in EOS_TOKENS`)** اگر کلمه تولید شده یک خط جدید یا توکن پایان متن (`<|endoftext|>`)بود، حلقه متوقف می‌شود.

```
[10]: EOS_TOKENS = [
          198, # newline
          50256 # tokenizer.eos_token_id = '<|endoftext|>'
      ]

      def generate(model, tokenizer, text, max_new_tokens=100, do_sample=False, top_k=None, verbose=True):
          input_ids = tokenizer(text, return_tensors='pt', return_attention_mask=False)['input_ids']
          if verbose:
              print(tokenizer.decode(input_ids[0]))
          for i in range(max_new_tokens):
              # run model
              output = model(input_ids = input_ids)

              next_token_logits = output.logits[:, -1, :]
              next_token = torch.argmax(next_token_logits, dim=-1).item()

              # add next_token to the input
              input_ids = torch.cat([input_ids, torch.tensor([[next_token]])], dim=1)
              if verbose:
                  print(tokenizer.decode([next_token]))
              # stop if generated newline or eos_token (end-of-sequence)
              if next_token in EOS_TOKENS:
                  break
          print(tokenizer.decode(input_ids[0]))
```

.6

این تصویر در واقع بخش) «Playground»**محیط آزمایش** (همان کد قبلی است. تفاوتش با تصویر قبل این است که در اینجا دیگر خبری از حلقه (`for loop`) برای تولید یک متن طولانی نیست؛ بلکه کد ساده شده تا **فقط و فقط یک کلمه (توکن) بعدی** را پیش‌بینی کند.

خلاصه کارکرد این خطوط به این شرح است:

1. `input_ids = tokenizer(...)`: متن ورودی شما را می‌گیرد و به شناسه‌های عددی (توکن) تبدیل می‌کند تا برای مدل قابل فهم شود.
2. `output = model(...)`: این اعداد را به مدل هوش مصنوعی می‌دهد تا نمرات خام (Logits) را برای کلمات بعدی محاسبه کند.
3. `next_token_logits = output.logits[:, -1, :]`: از میان کل نتایج، نمرات مربوط به **آخرین کلمه ورودی** را بیرون می‌کشد (چون کلمه بعد از آن برای ما مهم است).
4. `next_token = torch.argmax(...).item()`: بزرگ‌ترین نمره را در میان واژه‌نامه پیدا کرده و شناسه عددی آن کلمه (مثلاً یک عدد مثل `314`) را به عنوان کلمه بعدی استخراج می‌کند.

The process of exercises :

Exercise 1 – Greedy Generation

In the first exercise, I implemented greedy decoding for text generation. The objective was to complete the generate() function so that the model always selects the token with the highest prediction score at each generation step.

After passing the input text to GPT-2, the model produces a tensor called output.logits, which contains raw prediction scores for every token in the vocabulary. To generate the next token, I extracted the logits corresponding to the last position of the sequence using:

next_token_logits = output.logits[:, -1, :]

The next token was then selected using:

next_token = torch.argmax(next_token_logits, dim=-1).item()

The argmax operation chooses the token with the highest score. This process was repeated iteratively inside a loop, appending each newly generated token to the input sequence until a stop token or the maximum number of tokens was reached.

When generating text from the prompt:

"The name of the president of USA is"

the model produced:

"The name of the president of USA is "President Obama.""

This exercise demonstrated how autoregressive language models generate text by repeatedly selecting the most probable next token. Greedy decoding produces deterministic outputs and tends to generate coherent text, but it lacks diversity because it always chooses the single most likely option.

Exercise 2 – Random Sampling

In the second exercise, I extended the generation function to support probabilistic sampling. Instead of always selecting the highest-scoring token, the model was allowed to sample tokens according to their predicted probabilities.

First, the logits were converted into probabilities using the softmax function:

probs = F.softmax(next_token_logits, dim=-1)

Then, the next token was sampled from the probability distribution using:

next_token = torch.multinomial(probs, num_samples=1).item()

Unlike greedy decoding, this approach introduces randomness into the generation process. Tokens with higher probabilities are still more likely to be selected, but less probable tokens can also be chosen.

Using the same prompt, the generated output differed significantly from the greedy result. For example, the model generated text beginning with:

"The name of the president of USA is Warren Buffett..."

This demonstrates the increased diversity of random sampling. However, it also shows one of its disadvantages: the model may generate less accurate or less coherent continuations because it can sample from a very large vocabulary, including relatively unlikely tokens.

The main lesson from this exercise is that sampling increases creativity and variability but may reduce factual accuracy and consistency.

Exercise 3 – Top-k Sampling

In the third exercise, I implemented top-k sampling to improve the quality of probabilistic generation. The goal was to restrict sampling to only the k most likely candidate tokens before performing random selection.

The top-k candidates were obtained using:

```
topk = torch.topk(next_token_logits, k=top_k, dim=-1)
```

This function returns both the scores (topk.values) and the corresponding token indices (topk.indices) for the k highest-scoring tokens.

The probabilities were then computed only over these top-k candidates:

```
probs = F.softmax(topk.values, dim=-1)
```

A token was sampled from this reduced distribution:

```
sampled_idx = torch.multinomial(probs, num_samples=1)
```

Finally, the sampled position was mapped back to the original vocabulary index:

```
next_token = topk.indices.gather(-1, sampled_idx).item()
```

Using top_k = 10, the model generated:

"The name of the president of USA is "The Donald." He is a very conservative Republican..."

Although the generated statement is not necessarily factually correct, the output was generally more coherent than the unrestricted sampling used in Exercise 2.

Top-k sampling provides a balance between deterministic generation and unrestricted randomness. It preserves diversity by allowing multiple possible continuations while reducing the chance of selecting highly unlikely tokens. As a result, it often produces more natural and fluent text than standard sampling.

Conclusion

This assignment demonstrated three common text generation strategies used in autoregressive language models:

Greedy Decoding selects the most probable token at every step, producing deterministic and consistent outputs.

Random Sampling generates more diverse text by sampling from the entire probability distribution but may introduce errors and incoherent continuations.

Top-k Sampling restricts sampling to the most likely candidates, achieving a better balance between diversity and quality.

سوال آخر :

Temperature is a parameter that controls the randomness of the probability distribution used during sampling. Before applying the softmax function, the model logits were divided by a temperature value (T). The adjusted probabilities were then used to sample the next token.

Three different temperature values were tested:

Temperature = 0.7

When the temperature was set to 0.7, the probability distribution became sharper. High-probability tokens received even higher probabilities, while low-probability tokens became less likely to be selected. As a result, the generated text was more deterministic and closely followed the model's most likely continuation. The output was generally more coherent and predictable.

Temperature = 1.0

A temperature of 1.0 corresponds to the standard sampling procedure. The original probability distribution produced by the model was preserved without modification. The generated text showed a balance between coherence and diversity.

Temperature = 1.5

When the temperature was increased to 1.5, the probability distribution became flatter. Tokens with lower probabilities became more likely to be selected, introducing greater randomness into the generation process. The resulting text was more diverse and creative, although it occasionally became less coherent and less focused.

Observations

The experiments demonstrated that temperature directly affects the diversity of generated text. Lower temperatures encourage the model to select highly probable tokens, producing safer and more predictable outputs. Higher temperatures increase randomness and creativity but may reduce overall text quality and coherence.

Assignment 4 :

What does Named Entity Recognition do?

NER is the process of automatically identifying and classifying key pieces of information (called "entities") in unstructured text into predefined categories.

GLiNER stands for **Generalist and Lightweight Model for Named Entity Recognition**.

It is an open-source, highly efficient Natural Language Processing (NLP) framework designed to overcome the strict limitations of traditional NER tools (like standard spaCy models) and the high costs of Large Language Models (LLMs like ChatGPT).

How It Works: The "Matching" Architecture

Unlike generative LLMs that generate text word-by-word (which is slow), GLiNER treats NER as a **matching problem** in a shared space.

1. **The Input:** You give GLiNER the text *and* a list of custom labels you want to find.
    - *Example Text:* "We used Python to analyze data on the Apple MacBook."
    - *Example Labels:* ["Programming Language", "Hardware"]
2. **The Shared Latent Space:** GLiNER processes the text and your labels simultaneously. It projects the text chunks (spans) and the labels into a shared mathematical space.
3. **The Similarity Match:** It calculates a matching score between the vector of a word span (like *"Python"*) and the vector of a label (like *"Programming Language"*). If the score is high, it extracts it.